دليل الحل الهندسي والتطبيقي المتكامل لتكليف المحاضرة الرابعة: Student Controller Lifecycle

تطبيق معمارية دورة الحياة الكاملة (CRUD)، البحث، الفلترة، التقسيم (Pagination)، ومقال لينكد إن التخصصي (AutoMapper & HTTP Status Codes)، مع التحليل الهندسي المعمق لـ async/await وتقرير الـ Self-Study الشامل.

اسم الطالب (Student Name):

زياد محمد إمام محمود (Zeyad Mohamed Emam)

المجموعة الدراسية (Course Group):

DotNet With Elwakeel ▪ Banha Group

بيئة العمل والتطوير (Tech Stack):

.NET 8.0 ▪ EF Core 8.0 ▪ C# 12 ▪ Bootstrap 5

تاريخ التسليم (Submission Date):

أكتوبر 2026

فهرس المحتويات والمحاور الهندسية (Table of Contents)

1. **المحور الأول:** دورة حياة الـ Student Controller الكاملة (Task 1: Full Business Lifecycle)
2. **المحور الثاني:** مقال لينكد إن التخصصي (Task 2: LinkedIn Article: AutoMapper & HTTP Status Codes)
3. **المحور الثالث:** المعمارية العميقة للكلمات المفتاحية async و await (Task 3: Purpose of async/await)
4. **المحور الرابع:** تقرير الدراسة الذاتية الشامل (Task 4: Self-Study Report - 4 Comprehensive Topics)
5. **المحور الخامس:** استعراض الشفرة المصدرية وصفحات الـ Razor (Source Code & Views Implementation)

Task 1: Student Controller

1 — دورة حياة الـ Student Controller الكاملة (Task 1: Full Lifecycle)

المطلوب في التاسك الأول بناء دورة حياة الأعمال الكاملة **(Full Business Life Cycle)** للـ **Student Controller** معتمداً على نموذج الفئات السابق تصميمه، ومحققاً المتطلبات الهندسية الخمسة:

- توفير الأفعال الأساسية: `GetAll()`, `GetById()`, `Add`, `Edit`, `Delete`.
- عند إضافة أو تعديل طالب (Add / Edit) يلزم عرض كافة الأقسام **(All Departments)** داخل قائمة منسدلة `<select>` ليختار المستخدم منها بكل سهولة.
- عند طلب الحذف (Delete)، يتم التوجيه إجبارياً إلى صفحة تحذيرية **(Warning View)** تطلب التأكيد الصريح قبل المسح من قاعدة البيانات (لحماية تجربة المستخدم ومنع الحذف الخطأ).
- في صفحة العرض العام `GetAll`، تطبيق ثلاث خواص متقدمة: **التقسيم لصفحات (Pagination)**، **البحث باسم الطالب (Search by name)**، و**الفلترة بحسب القسم (Filter by Department)**.

1.1 المعمارية الهندسية واستخدام الـ ViewModels

من أهم القواعد البرمجية النظيفة التي تعلمناها مع البشمهندس عبدالرحمن الوكيل: **عدم تمرير الـ Domain Entity مباشرة إلى الـ View عندما تحتاج الصفحة بيانات مركبة من أكثر من مصدر**.

1. StudentWithDeptListViewModel

يُستخدم في عمليتي **Add** و **Edit**. يحمل بيانات الطالب المطلوب إضافته أو تعديله (Name, Age, Address, DeptId) بالإضافة إلى قائمة `List<Department> DeptList` لتغذية عنصر الـ Dropdown.

2. StudentIndexViewModel

يُستخدم في عملية **GetAll**. يدمج قائمة الطلاب في الصفحة الحالية، وقائمة الأقسام، ومعايير الفلترة (Search, SelectedDeptId)، وبيانات ترقيم الصفحات (CurrentPage, TotalPages, TotalCount).

1.2 جدول مسارات وأفعال الـ Student Controller ودورات الحياة

Action Name	HTTP Method	المدخلات (Parameters)	المسؤولية ودورة الحياة (Lifecycle Role)	نوع النتيجة / View
`GetAll / Index`	`GET`	`search, deptId, page, pageSize`	استقبال معايير البحث والفلترة وتطبيق معادلة الـ Pagination، وإرجاع الـ ViewModel.	`View("GetAll", VM)`
`GetById / Details`	`GET`	`int id`	جلب بيانات الطالب بالـ ID مع بيانات قسمه (Eager Loading)، أو إرجاع `NotFound()`.	`View("Details", Student)`
`Add`	`GET`	—	الخطوة 1 للإضافة: فتح شاشة الإدخال فارغة	`View("Add", VM)`

Action Name	HTTP Method	المدخلات (Parameters)	المسؤولية ودورة الحياة (Lifecycle Role)	نوع النتيجة / View
			ومحملة بكل الأقسام في القائمة المنسدلة.	
`SaveAdd`	POST	`Student student`	الخطوة 2 للإضافة: استقبال البيانات عبر Model Binding، حفظها بقاعدة البيانات، ثم Redirect.	`RedirectToAction("GetAll")`
`Edit`	GET	`int id`	الخطوة 1 للتعديل: جلب السجل القديم وتعبئة الـ Dropdown مع تحديد قسم الطالب الحالي.	`View("Edit", VM)`
`SaveEdit`	POST	`Student student, int id`	الخطوة 2 للتعديل: تحديث الحقول في الـ Context وحفظ التغييرات، ثم Redirect.	`RedirectToAction("GetAll")`
`Delete`	GET	`int id`	**Warning View:** عرض كارت أحمر تحذيري ببيانات الطالب وسؤال المستخدم: هل أنت متأكد؟	`View("Delete", Student)`
`ConfirmDelete`	POST	`int id`	التنفيذ الفعلي: مسح الكيان من الـ DbSet ثم حفظ الـ Context، وإعادة التوجيه لـ GetAll.	`RedirectToAction("GetAll")`

ملاحظة هندسية حول الـ Resiliency (مرونة النظام): تم بناء طبقة `StudentBL` بحيث تحاول تنفيذ العمليات على قاعدة البيانات عبر `MsaooDbContext`، وفي حالة عدم توفر خادم SQL Server محلياً (كما هو الحال في بعض بيئات الاختبار أو الماك)، تعتمد تلقائياً على **In-Memory Mock Repository** لتظل الشاشات والوظائف تعمل بنسبة 100% بدون أي Crash.

Task 2: LinkedIn Article

2 — مقال لينكد إن التخصصي (Task 2: AutoMapper & HTTP Status Codes)

المطلوب في التاسك الثاني صياغة مقال تخصصي موجه لمجتمع المطورين على **LinkedIn** يناقش مكتبة **AutoMapper** وأهميتها في تسريع عملية التحويل بين الكيانات، مع تغطية القيمة المضافة **(Bonus Plus)** لأهمية ضبط واستخدام الـ **HTTP Response Status Codes** في بناء تطبيقات الويب والـ APIs الحديثة.

2.1 نص المقال الجاهز للنشر المباشر على LinkedIn (باللغة العربية والإنجليزية المهنية)

عنوان البوست المقترح: 🚀 رحلة المهندس بين Manual Mapping و AutoMapper.. ولماذا يجب أن تتوقف عن إرجاع 200 OK لكل شيء؟! 💡

LinkedIn Post Content (Ready to Copy & Publish)

عمرك عديت كم سطر كود كتبته في حياتك مجرد:
dto.Name = model.Name;
dto.Age = model.Age;
dto.Address = model.Address; ؟ 😅

في مشاريع الـ ASP.NET Core MVC و الـ Web APIs، دايماً بنفصل بين طبقة الـ Database (Domain Models) ولتقليل (Security) لحماية النظام Frontend (ViewModels أو DTOs) وبين البيانات اللي رايحة للـ استهلاك الـ Network.
لكن التحدي الأكبر اللي بيقابل المطور يومياً هو: إزاي ننقل البيانات بين الكائنين بأقل مجهود وبأعلى كفاءة؟

ناقشنا بعمق موضوعين، خلال تدريبنا العملي في كورس الـ MVC مع البشمهندس Abdelrahman Shaaban Elwakeel من أهم مهارات مهندس الـ Backend:

1️⃣ الحل السحري: AutoMapper vs Manual Mapping
📌 الـ Manual Mapping ميزته إنه Type-safe وسريع جداً في الـ Execution وواضح في الـ Debugging، لكن عيبه الأكبر إنه بيتحول لـ Boilerplate Code ممل ومستهلك للوقت، وممكن تنسى تعيّن خاصية جديدة ضفتها للـ Model!
📌 هنا بيجي دور AutoMapper:
بنقول: Service أو الـ Controller وبسطر واحد داخل الـ Mapping Profile، بسطر واحد بنعرف الـ
var studentVM = _mapper.Map<StudentDetailsVM>(student);

والأجمل من كده ميزة الـ Flattening:
ذكي كفاية إنه يربطها تلقائياً بـ الـ AutoMapper لو عندك جوه الطالب Student.Department.Name، داخل الـ ViewModel بدون أي Configuration إضافي! اسمها DepartmentName
💡 Pro Tip: مع EF Core استخدم دايماً `ProjectTo<T>()` علشان الـ SQL Query يتولد فيه فقط الأعمدة المطلوبة للـ DTO (SQL Projection) وماتسحبش كل أعمدة الجدول في الميموري!

2️⃣ القيمة المضافة (Bonus Plus): ثقافة الـ HTTP Response Status Codes 🚦
إن الـ: المحترف والمبتدئ بين المطور بتفصل اللي الشائعة الأخطاء من أكبر API يرجع دايماً Status 200 OK حتى لو حصل خطأ أو المورد مش موجود! 🤦

هي لغة التفاهم العالمية بين السيرفر والمتصفح/الموبايل HTTP Status Codes الـ:
✅ 200 OK: للعمليات القراءة والتحديث الناجحة.
✅ 201 Created: لما تضيف مورد جديد (Add Student). الأصول الهندسية تقتضي إنك ترجع معاه الـ Header اسمه Location فيه رابط المورد الجديد `CreatedAtAction`.

✅ 204 No Content: لما تنفذ عملية Delete أو Update ومش محتاج ترجع Data في الـ Body.
⚠️ 400 Bad Request: لما تكون المدخلات غير صحيحة أو الـ Model Validation يفشل.
🔒 401 Unauthorized: المستخدم مش عامل Login أصلاً (Missing Identity).
🚫 403 Forbidden: المستخدم مسجل لكن معندوش الـ Role يسمحله بالحذف (Forbidden Access).
🔍 404 Not Found: الـ ID اللي طالبه مش موجود في قاعدة البيانات.
💥 500 Internal Server Error: حصل Exception غير متوقع في السيرفر.

باستخدامك للـ Status Codes الصحيحة بيسهل حياة مهندسي الـ Frontend، بيخلي تطبيقك RESTful، وبيخلي الـ Logging Tools و الـ Monitoring تشتغل بدقة متناهية.

سؤالي للمهندسين الزملاء:
وهل بتعتمد(Mapster أو مكتبات تانية زي) Manual Mapping ولا AutoMapper في مشاريعك الحالية، بتفضل بدقة في مشاريعك؟ شاركوني تجاربكم في التعليقات على Status Codes! 👇

#DotNet #CSharp #ASPNETCore #MVC #WebAPI #CleanCode #AutoMapper #SoftwareEngineering #BackendDevelopment #Architecture

2.2 جدول المقارنة الفنية: AutoMapper مقابل Manual Mapping مقابل Mapster

المعيار الهيكلي	Manual Mapping	AutoMapper	Mapster (البديل الحديث)
سرعة التنفيذ (Execution Speed)	الأسرع مطلقاً (0 overhead)	سريع (يعتمد على IL Emitting بعد الـ Warmup)	فائق السرعة (يقارب الـ Manual)
جهد الصيانة والـ Boilerplate	مرتفع جداً (كتابة يدوية لكل حقل)	منخفض جداً (سطر واحد)	منخفض جداً
Compile-Time Safety	آمن 100% في وقت الترجمة	يحتاج اختبارات `AssertConfigurationIsValid`	آمن عبر Code Generation
تكامل EF Core Projection	يدوي عبر `.Select(s => new ...)`	`.ProjectTo<VM>()`	`.ProjectToType<VM>()`

Task 3: Concurrency & Async

3 — الغرض والمعمارية العميقة لـ async و await (Task 3: Async/Await Purpose)

المطلوب في التاسك الثالث الإجابة التفصيلية عن الغرض من الكلمات المفتاحية **async** و **await** في منصة .NET، وكيف تؤثر جذرياً على أداء وقابلية التوسع **(Scalability)** لتطبيقات الويب.

3.1 الغرض الأساسي: حل أزمة مجاعة الخيوط (Thread Pool Starvation)

في تطبيقات الـ Web التي تستقبل آلاف الطلبات المتزامنة (Concurrent Requests)، يخصص خادم الويب (مثل Kestrel أو IIS) خيط معالجة **(Thread)** من مخزن الخيوط **(Thread Pool)** لخدمة كل طلب قادم.

إذا كان الكود تزامني **Synchronous**: عندما يستدعي التطبيق قاعدة البيانات لقراءة بيانات 1000 طالب عبر `_context.Students.ToList()`، فإن خيط المعالجة يدخل في حالة تجميد تام **(Thread Blocked)** لمدة 100-300 مللي ثانية في انتظار عودة الشبكة بالبيانات. خلال هذا الوقت، يستهلك الخيط ذاكرة (نحو 1 ميجابايت Stack Space) ولا يقدم أي فائدة حسابية للـ CPU!

سيناريو الانهيار (The Thread Pool Starvation Trap): إذا وصل 100 طلب متزامن وجميعها مجمدة في انتظار قواعد البيانات، تنفد خيوط الـ Thread Pool بالكامل. الطلبات الجديدة تبدأ بالتراكم في الطابور (Queue Delay)، مما يؤدي لارتفاع زمن الاستجابة (High Latency) وفي النهاية ظهور أخطاء **HTTP 503 Service Unavailable** وانهيار السيرفر.

3.2 كيف تحل async و await هذه الكارثة؟ (Non-Blocking Asynchronous I/O)

عند استخدام `await _context.Students.ToListAsync()`، يحدث ما يلي خطوة بخطوة:

1. يرسل خيط المعالجة أمر القراءة إلى نظام التشغيل وكارت الشبكة عبر تقنية تسمى **I/O Completion Ports (IOCP)** في نظام Windows أو **epoll** في Linux.
2. فور إرسال الأمر، **يتحرر الخيط فوراً (Thread Released)** ويعود إلى الـ Thread Pool ليخدم مستخدمين آخرين! لا يوجد أي خيط مجمد في الذاكرة.
3. عندما تنتهي قاعدة البيانات وترسل حزم البيانات عبر الشبكة، يصدر كارت الشبكة إشارة مقاطعة للأجهزة (Hardware Interrupt)، ويقوم الـ OS بتنبيه مكتبة الـ Task Runtime.
4. يقوم الـ Thread Pool باستدعاء أي خيط متاح لاستكمال بقية الكود (Continuation) من النقطة التي توقف عندها بعد الـ `await`!

3.3 السحر الداخلي لمترجم C# (Under the Hood: The State Machine)

الكلمات `async` و `await` ليست مجرد سكر تركيبي (Syntactic Sugar)، بل هي تعليمة مباشرة لمترجم **Roslyn Compiler** لإعادة تشكيل الدالة بالكامل!

يقوم المترجم بإنشاء هيكل خفي **(Hidden Struct / Class)** يطبق الواجهة `IAsyncStateMachine`، ويقسم الدالة الأصلية إلى حالات مرقمة (State 0, State 1, State -1). عند الوصول لأمر `await`:

- إذا كانت المهمة مكتملة بالفعل، يكمل الكود سيره دون مغادرة الخيط (Fast Path).
- إذا كانت المهمة غير مكتملة، يسجل الكود دالة الإكمال (Awaiter Callback)، ويغير رقم الحالة (State)، ثم يخرج فوراً مفسحاً المجال للخيط!

C# Async/Await vs Compiler State Machine Representation

```
// الكود المكتوب بواسطة المطور:
public async Task<IActionResult> GetAll()
{
    var data = await _studentBL.GetAllAsync();
    return View(data);
}

// مفهوم مبسط لـ State Machine) ما يولده المترجم تحت الغطاء):
[CompilerGenerated]
struct <GetAll>d__0 : IAsyncStateMachine
{
    public int <>1__state;
    public AsyncTaskMethodBuilder<IActionResult> <>t__builder;
    private TaskAwaiter<List<Student>> <>u__1;

    public void MoveNext()
    {
        // التبديل بين الحالات وإدارة دورة حياة الـ Task
    }
}
```

3.4 القواعد الذهبية وأشهر الأخطاء الشائعة (Golden Rules & Anti-Patterns)

الممارسة الخاطئة (Anti-Pattern)	الممارسة الصحيحة (Best Practice)	السبب الهندسي
`public async void DoWork()`	`public async Task DoWorkAsync()`	دوال `async void` تمنع التقاط الـ Exceptions وتؤدي لانهيار العملية (Crash). تُستخدم فقط في UI Events.
استخدام `task.Result` أو `task.Wait()`	استخدام `await task` دائماً	خلط الـ Sync مع الـ Async يسبب تجميد خيوط المعالجة وحدوث **Deadlocks** خطيرة.
تطبيق `Task.Run()` لعمليات DB I/O	استخدام دوال I/O الأصلية كـ `ToListAsync()`	`Task.Run()` يحجز خيطاً إضافياً من الـ Pool بلا داعٍ لعملية I/O لا تحتاج معالجة حسابية.
إهمال الـ `CancellationToken`	تمرير `cancellationToken` لجميع دوال EF	لو أغلق المستخدم المتصفح، يتم إلغاء كويري قاعدة البيانات فوراً وتوفير موارد السيرفر.

Task 4: Self Study

4 — تقرير الدراسة الذاتية الشامل (Task 4: Self-Study Report)

المطلوب في التاسك الرابع إعداد تقرير دراسة ذاتية **(Self-Study Report)** شامل يغطي أهم الموضوعات الهندسية التي طرحها البشمهندس عبدالرحمن الوكيل خلال المحاضرة وفي تعليقات الأكواد:

4.1 استراتيجيات تحميل البيانات في EF Core مقابل EF 6 (Loading Strategies)

في تطبيقات الـ ORM، تمثل إدارة العلاقات بين الجداول (مثل علاقة الطالب بالقسم) أحد أخطر عوامل تحديد أداء واستقرار النظام. توجد ثلاث استراتيجيات رئيسية:

1. التحميل المسبق المباشر (Eager Loading)

جلب الكيان الأصلي والكيانات المرتبطة به في نفس استعلام الـ SQL أو عبر استعلامات متزامنة باستخدام دالتي `.Include()` و `.ThenInclude()`.

الميزة: رحلة واحدة للشبكة وتفادي مشكلة الـ N+1 Queries.

في EF Core: تدعم تقنية `AsSplitQuery()` لمنع انفجار البيانات الناتج عن الـ Cartesian Explosion في جداول الـ Many-to-Many!

2. التحميل الصريح المنضبط (Explicit Loading)

جلب الكيان الأصلي أولاً، ثم في مرحلة لاحقة وبأمر برمجي صريح من المطور يتم طلب تحميل العلاقة باستخدام `context.Entry(entity).Reference(...).Load()` للعلاقات الفردية أو `Collection(...).Load()` للمجموعات.

الميزة: تحكم دقيق جداً في توقيت وحجم البيانات المحملة وفقاً لشروط منطق الأعمال.

3. التحميل الكسول التلقائي (Lazy Loading): الفرق التاريخي بين EF 6 و EF Core

في **EF 6 (قبل Core)**، كان الـ Lazy Loading هو السلوك الافتراضي بمجرد وضع كلمة `virtual` على الـ Navigation Property. كان هذا السلوك يتسبب في كارثة الأداء الشهيرة **(The N+1 Query Problem)**: إذا قرأت 100 طالب في لوب وعرضت اسم قسم كل طالب، يرسل السيرفر استعلاماً واحداً لجلب الطلاب ثم 100 استعلام منفصل إضافي لجلب أقسامهم!

لذلك، في **EF Core (بعد Core)**، قرر مهندسو مايكروسوفت **إلغاء تفعيل الـ Lazy Loading افتراضياً**. وإذا أراد المطور تفعيله، يجب عليه تثبيت حزمة مستقلة `Microsoft.EntityFrameworkCore.Proxies` واستدعاء `UseLazyLoadingProxies()` صراحة، مما يجبر المهندس على التفكير الواعي في استراتيجية استعلاماته.

4.2 نمط التصميم Builder Design Pattern في معمارية .NET 8 الحديثة

عند الانتقال من .NET Core القديم إلى .NET 6 و .NET 8، تساءل الجميع عن سر تبسيط كود الـ `Program.cs` واستخدام:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

السبب هو التطبيق المباشر لنمط التصميم الشهير **Builder Design Pattern** (أحد أنماط الـ Creational Patterns من عصابة الـ GoF):

- **فصل مرحلة التكوين (Configuration) عن مرحلة التشغيل (Runtime Execution):** يسمح الـ Builder بإضافة خدمات الـ Dependency Injection داخل `builder.Services`، وضبط إعدادات الـ Logging والـ Configuration بكل مرونة وبشكل تدريجي.
- **مبدأ التجميد وثبات الحالة (Immutability):** بمجرد استدعاء دالة `builder.Build()`، يتم بناء حاوية الخدمات النهائية (Service Provider) وتجميدها تماماً. لا يمكن لأي كود لاحق إضافة أو حذف خدمات أثناء تشغيل الـ Web Application،

مما يحمي النظام من أخطاء الـ Thread-Safety والـ Race Conditions.

- **بناء خط الأنابيب (Middleware Pipeline):** كائن `app` الناتج يتولى فقط ضبط مسارات الطلبات وترتيب الـ Middlewares مثل `app.UseRouting()` و `app.UseAuthorization()` ثم بدء الاستماع للطلبات عبر `app.Run()`.

4.3 معمارية الـ Controller Factory ودورة حياة الكنترولر في ASP.NET Core MVC

عندما يطلب المتصفح مساراً مثل `/Student/GetAll`، كيف يفهم الـ Framework هذا الطلب وكيف يُنشئ كائن الـ Controller؟

1. **Routing Middleware:** يقوم بمطابقة الـ URL مع قوالب التوجيه المسجلة واستخراج اسم الـ Controller (وهو "Student") واسم الـ Action (وهو "GetAll").
2. **مصنع الكنترولر (IControllerFactory):** المكون الافتراضي المسجل في النظام هو `DefaultControllerFactory`. يقوم هذا المصنع بفحص الـ Assembly للعثور على الفئة المناسبة بناءً على الشروط القياسية: (أن تنتهي بكلمة Controller، ترث من Controller، وتكون فئة عامة Public وغير مجردة وغير Generic).
3. **منشط الكنترولر (IControllerActivator):** يقوم بإنشاء نسخة فعلية (Instance) من الكنترولر، وسحب كافة الـ Dependencies المطلوبة من حاوية الـ Dependency Injection وتمريرها في الـ Constructor (Constructor Injection).
4. **تنفيذ الـ Action:** يقوم الـ Model Binder باستخراج المدخلات، وتعمل الـ Action Filters، ثم يتم تنفيذ دالة الـ Action.
5. **التخلص من الكائن (Disposal):** عند اكتمال الـ Response، يقوم الـ Controller Factory باستدعاء `ReleaseController`، فإذا كان الكنترولر يطبق `IDisposable` أو `IAsyncDisposable`، يتم التخلص منه ومن موارده فوراً من الذاكرة لضمان عدم حدوث تسريب ذاكرة (Memory Leak).

4.4 التجريد مقابل التخصيص: المقارنة بين IActionResult و ActionResult<T>

في بدايات الـ MVC، كان المطور يعيد أنواعاً محددة (Concrete Types) مثل `ViewResult` أو `JsonResult`. ولكن في التطبيقات الحديثة:

- **واجهة IActionResult:** تمثل أقصى درجات التجريد (Abstraction). تسمح للـ Action بإرجاع ردود مختلفة بحسب مسار منطق الأعمال (مثل إرجاع `View()` إذا وُجد الطالب، أو إرجاع `NotFound()` إذا كان الـ ID خطأ، أو إرجاع `Content()`).
- **النوع ActionResult<T>:** تم تقديمه في ASP.NET Core 2.1 خصيصاً لخدمة الـ Web APIs و Swagger/OpenAPI. يسمح بتوفير مرونة الـ `IActionResult` مع الحفاظ على الـ Strong-Typing للنوع المرجع `T` ليتمكن الـ Documentation Generator والـ Frontend Client من معرفة شكل الـ Data Model بدقة.

Implementation Showcase

5 — الشفرة المصدرية المطبقة وصفحات الـ Razor (Source Code Implementation)

تم تطبيق الحل الكامل داخل المشروع بمساراته النموذجية وفقاً لأحدث ممارسات ASP.NET Core 8.0:

5.1 كود الـ Controller الأساسي (Controllers/StudentController.cs)

Controllers/StudentController.cs

```
public class StudentController : Controller
{
    private readonly StudentBL studentBL = new StudentBL();
    private readonly DepBL depBL = new DepBL();

    // 1- GetAll مع Pagination والبحث والفلترة
    public IActionResult GetAll(string? search, int? deptId, int page = 1, int pageSize = 4)
    {
        int totalCount;
        var students = studentBL.GetFiltered(search, deptId, page, pageSize, out totalCount);
        var depts = depBL.GetAll();
        var totalPages = (int)Math.Ceiling((double)totalCount / pageSize);

        var viewModel = new StudentIndexViewModel
        {
            Students = students,
            DeptList = depts,
            Search = search,
            SelectedDeptId = deptId,
            CurrentPage = page,
            PageSize = pageSize,
            TotalCount = totalCount,
            TotalPages = totalPages < 1 ? 1 : totalPages
        };
        return View("GetAll", viewModel);
    }

    // 2- GetById / Details
    public IActionResult GetById(int id)
    {
        var student = studentBL.GetById(id);
        if (student == null) return NotFound();
        return View("Details", student);
    }

    // 3- Add: تحميل الأقسام للـ Dropdown
    [HttpGet]
    public IActionResult Add()
    {
        var vm = new StudentWithDeptListViewModel { DeptList = depBL.GetAll() };
        return View("Add", vm);
    }
```

```
    [HttpPost]
    public IActionResult SaveAdd(Student student)
    {
        if (!string.IsNullOrWhiteSpace(student.Name))
        {
            studentBL.Add(student);
            return RedirectToAction(nameof(GetAll));
        }
        return View("Add", new StudentWithDeptListViewModel { DeptList = depBL.GetAll() });
    }

    // 4- Edit: تحميل بيانات الطالب والأقسام
    [HttpGet]
    public IActionResult Edit(int id)
    {
        var student = studentBL.GetById(id);
        if (student == null) return NotFound();
        var vm = new StudentWithDeptListViewModel
        {
            Id = student.Id, Name = student.Name, Age = student.Age,
            Address = student.Address, DeptId = student.DeptId, DeptList = depBL.GetAll()
        };
        return View("Edit", vm);
    }

    [HttpPost]
    public IActionResult SaveEdit(Student student, int id)
    {
        if (!string.IsNullOrWhiteSpace(student.Name))
        {
            studentBL.Update(id, student);
            return RedirectToAction(nameof(GetAll));
        }
        return View("Edit", student);
    }

    // 5- Delete: التوجيه لصفحة التحذير ثم التأكيد
    [HttpGet]
    public IActionResult Delete(int id)
    {
        var student = studentBL.GetById(id);
        if (student == null) return NotFound();
        return View("Delete", student); // Warning View
    }

    [HttpPost]
    public IActionResult ConfirmDelete(int id)
    {
        studentBL.Delete(id);
        return RedirectToAction(nameof(GetAll));
    }
}
```

5.2 كود صفحة العرض العام والفلترة (Views/Student/GetAll.cshtml)

Views/Student/GetAll.cshtml (Excerpt)

```
@model StudentIndexViewModel

<form method="get" action="/Student/GetAll" class="row g-3 align-items-end mb-4">
    <!-- Search Input -->
    <div class="col-md-5">
        <label class="form-label fw-bold">Search by Name</label>
        <input type="text" name="search" value="@Model.Search" class="form-control"
placeholder="Search student name..." />
    </div>

    <!-- Department Filter Dropdown -->
    <div class="col-md-4">
        <label class="form-label fw-bold">Filter by Department</label>
        <select name="deptId" class="form-select">
            <option value="">-- All Departments --</option>
            @foreach (var dept in Model.DeptList)
            {
                <option value="@dept.Id" selected="@(dept.Id ==
Model.SelectedDeptId)">@dept.Name</option>
            }
        </select>
    </div>

    <div class="col-md-3 d-flex gap-2">
        <button type="submit" class="btn btn-primary flex-grow-1">Filter</button>
        <a href="/Student/GetAll" class="btn btn-outline-secondary">Reset</a>
    </div>
</form>

<!-- Pagination Bar -->
<ul class="pagination">
    <li class="page-item @(!Model.HasPreviousPage ? "disabled" : "")">
        <a class="page-link" href="/Student/GetAll?
search=@Model.Search&deptId=@Model.SelectedDeptId&page=@(Model.CurrentPage - 1)">Previous</a>
    </li>
    @for (int p = 1; p <= Model.TotalPages; p++)
    {
        <li class="page-item @(p == Model.CurrentPage ? "active" : "")">
            <a class="page-link" href="/Student/GetAll?
search=@Model.Search&deptId=@Model.SelectedDeptId&page=@p">@p</a>
        </li>
    }
    <li class="page-item @(!Model.HasNextPage ? "disabled" : "")">
        <a class="page-link" href="/Student/GetAll?
search=@Model.Search&deptId=@Model.SelectedDeptId&page=@(Model.CurrentPage + 1)">Next</a>
    </li>
</ul>
```

خلاصة التقييم والتسليم: تم بحمد الله وتوفيقه استيفاء كافة متطلبات التكليف الرابع (Assignment 04) وفق أعلى المعايير الهندسية النظيفة، وتوثيق كافة الأسئلة المقالية والتقرير التخصصي داخل هذا المستند الشامل ليكون مرجعاً هندسياً متكاملاً.